

0.7.2. La même ligne, avec `do ... while` — doublé, deux positions

La boucle deux fois : la ligne 0 puis la ligne 5.

1. Ce qu'il faut comprendre

C'est le **0.7** et le **0.7.1 réunis** : la boucle du 0.7 (ligne 0), puis celle du 0.7.1 (ligne 5).

Deux boucles l'une après l'autre, avec la **même** variable `i` : déclarée une fois, elle est remise à 0 (`i = 0;`) avant la seconde boucle, sans `uint8_t` — on ne déclare pas deux fois le même nom.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **921 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
main()	74 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AdresseCase	10 o	← poser() – trouve l'adresse d'une case de l'écran
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles – la police entière

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

<code>poser()</code> × 2	37 o sur place, dans <code>main()</code>	+ <code>AdresseCase</code> (10 o)
<code>image()</code> × 1	3 o sur place, dans <code>main()</code>	+ <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

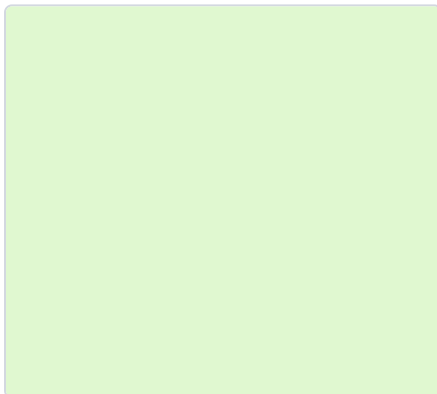
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  // 1. La ligne 0 (le 0.7)
  uint8_t i = 0;
  do {
    poser(i, 0, ALPHABET[i]); // la lettre n° i, colonne i, ligne 0
    i++;
  } while (i < 20);           // la condition est vérifiée À LA FIN
  // 2. La ligne 5 (le 0.7.1)
  i = 0;
  do {
    poser(i, 5, ALPHABET[i]); // la lettre n° i, colonne i, ligne 5
    i++;
  } while (i < 20);           // la condition est vérifiée À LA FIN

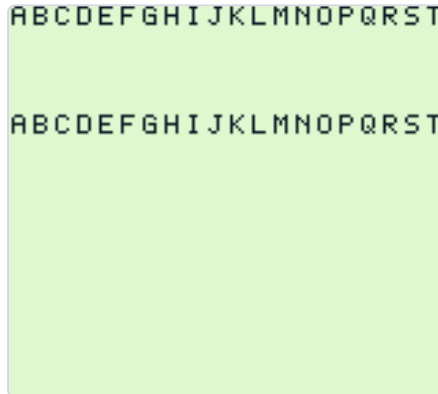
  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 0 et sur la ligne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 5 : A à T

